

50.044 System Security

Siddharth Ganesh
1006407

Lab 5 Report

Setup

The setup for this lab involved running the website's server locally using python.

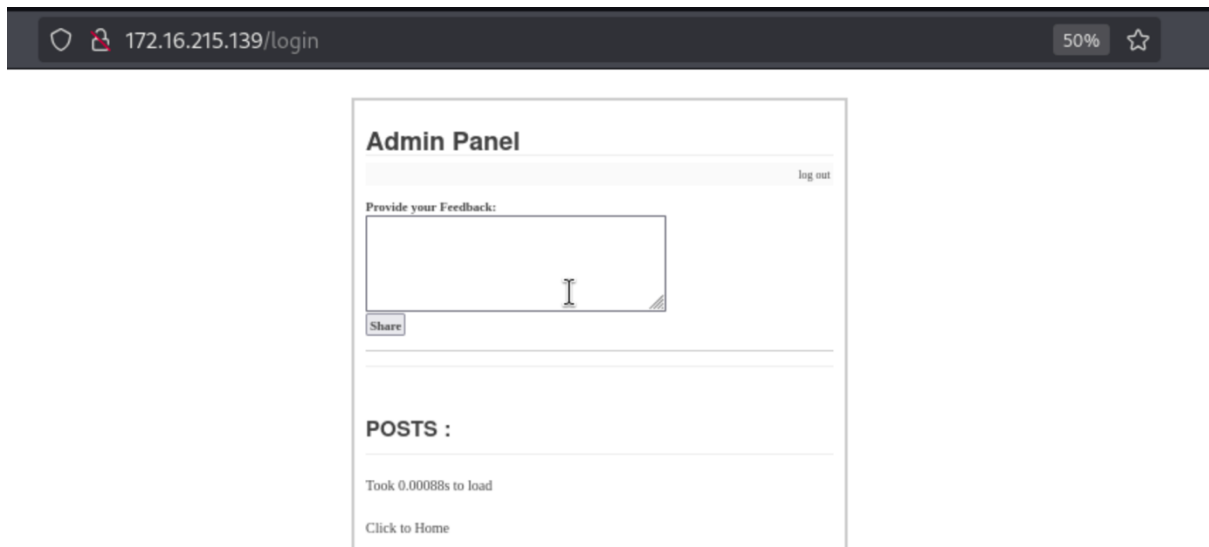
1. SQL Injection

After entering ' or 1=1 –into the login textbox, the SQL injection was successful, and I was granted access into the Admin account without a password:

A screenshot of a web application interface titled "Admin Panel". At the top right, there is a "log out" link. Below the title, there is a section labeled "Provide your Feedback:" with a large text input area and a "Share" button below it. Further down, there is a section titled "POSTS :". Below this, it says "Took 0.00447s to load" and "Click to Home".

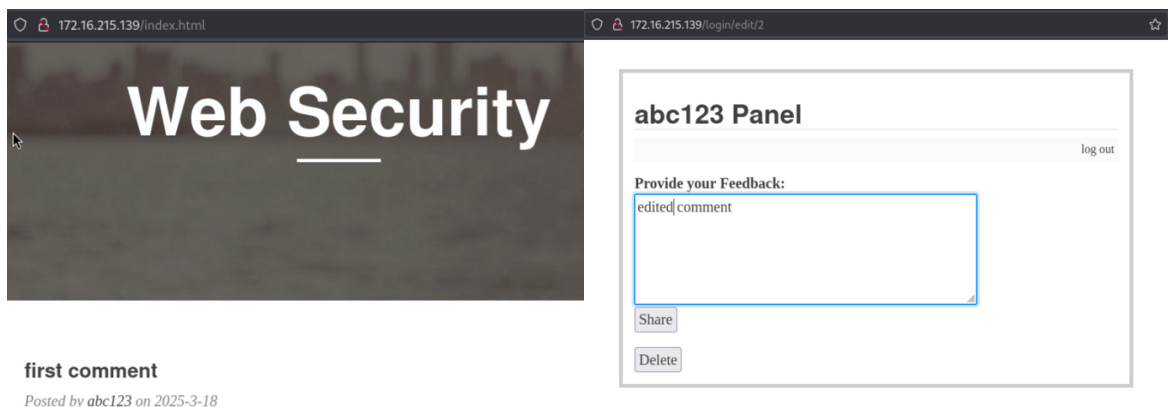
Another way to gain access to the Admin account via SQL injection would be to enter Admin' --. This converts everything after the apostrophe to a comment, allowing access into the Admin account:

A screenshot of a web application interface titled "Login Panel". At the top right, there is a "log in" link. Below the title, there are two blue message boxes: "Login Success!" and "You were logged out". Below these, there is a "Login" section with "Username:" and "Password:" labels. The username field contains the text "Admin' --". Below the password field, there are "Login" and "Register" buttons. At the bottom, there is a "Click to Home" link.

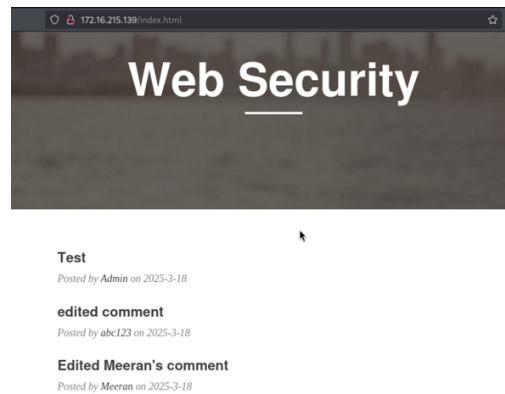
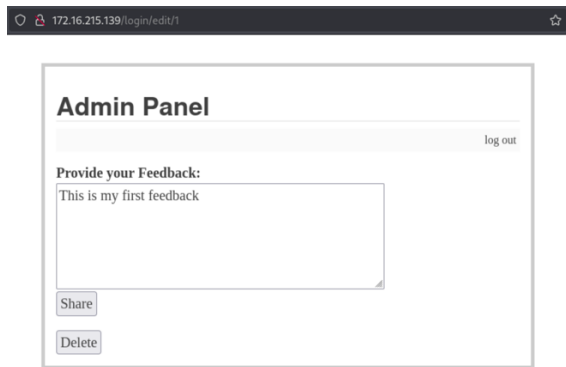


2. CSRF Attack

The comments functionality works as follows. After logging in, the user is able to post a comment through the Feedback textbox. When editing a comment, the user is redirected to a URL which identifies each comment by their ID in the database.



By changing this ID, I was able to access comments posted by other users. Changing this ID to 1 allowed me to edit the comment posted by Meeran from “This is my first feedback” to “Edited Meeran’s comment” as shown in the images below. The URL to change this comment was: <http://172.16.215.139/login/edit/1>

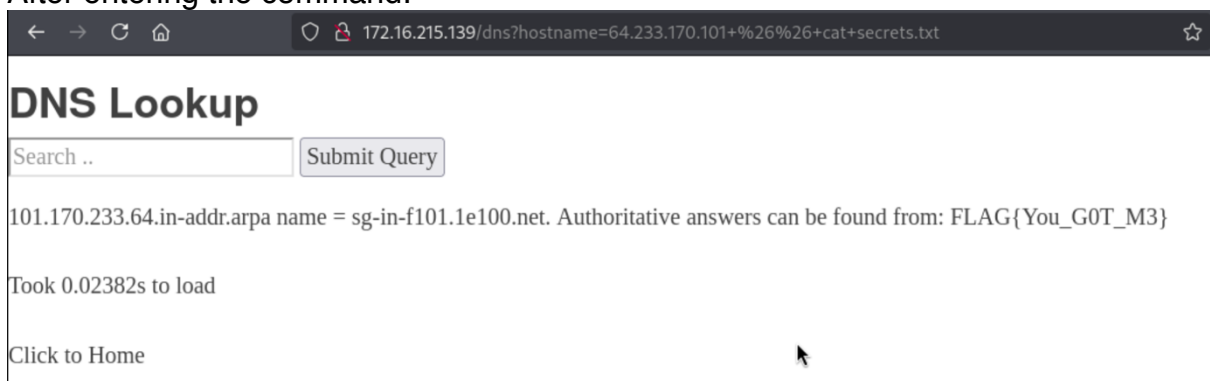


Edited to:

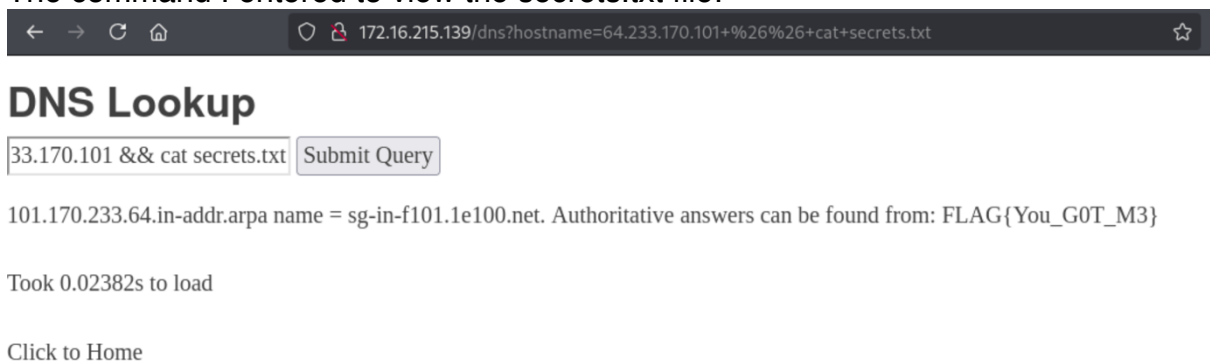
3. Command Line Injection

The lookup page allows entry of an address to get the results of its nslookup. The flaw here is that the inputs from this textbox are not sanitised for multiple commands. Hence, an attack is possible by inputting two commands through the && operator. Entering “64.233.170.101 && cat secrets.txt” allows us to view the secrets.txt:

After entering the command:

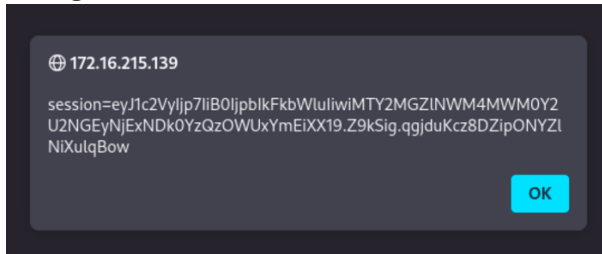


The command I entered to view the secrets.txt file:



4. XSS Attack

The input field in the comments page is vulnerable to XSS attacks. Entering `<script>alert(document.cookie)</script>` in it displays the current session cookie in a dialog box as well as in a comment:



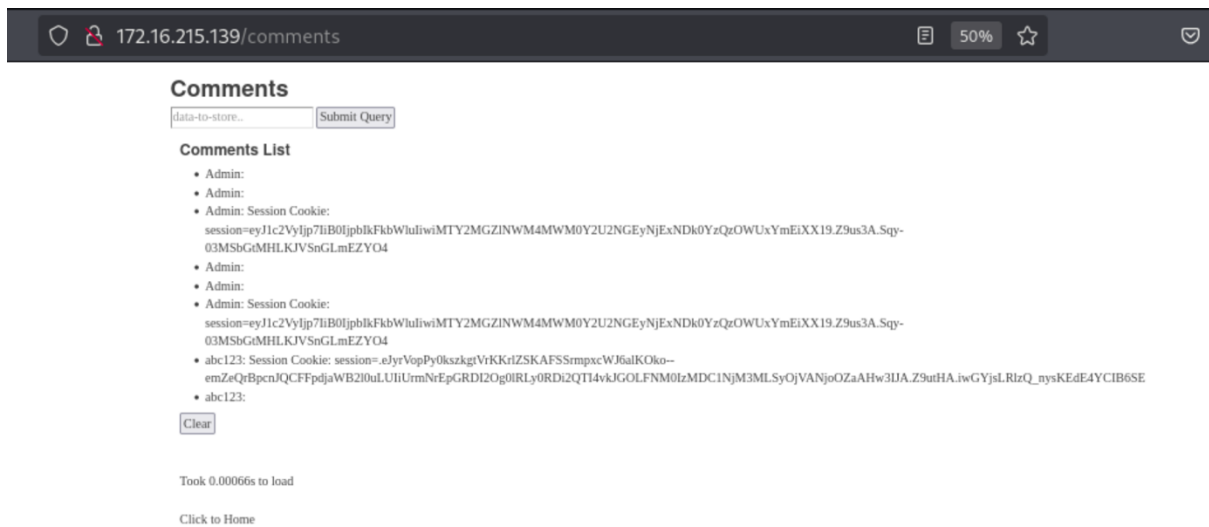
In order to automatically log the current user's session cookie as a comment as soon as they visit the comments page, I used the below code:

```
<script>
var cookie = document.cookie;
fetch('/comments?text=' + encodeURIComponent('Session Cookie: ' + cookie), {
  method: 'GET'
}).then(() => {
  fetch('/comments?text=' + encodeURIComponent('<script>var cookie =
document.cookie; fetch(\'/comments?text=\' + encodeURIComponent(\'Session
Cookie: \' + cookie), { method: \'GET\' });</script>'), {
    method: 'GET'
  });
});
</script>
```

After entering this code into the comments text box, whenever a user visits the comments page, their cookie will be automatically displayed as a comment:



After logging in as a different user (abc123), this user's session cookie is also visible as a comment:



Secure Code

Preventing SQL Injection

To prevent the possibilities of SQL injection attacks on the website, we should use parameterized queries instead of string concatenation in SQL commands. This prevents SQL Injection because user input is treated as data rather than SQL code. An example of a fix for the login function would be:

```
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = hash(request.form['password']) # Still needs
bcrypt

        # Secure query using parameterized SQL
        cur = g.db.cursor()
        cur.execute("SELECT * FROM users WHERE username=? AND
password=?", (username, password))
        user = cur.fetchone()

        if user:
            session['user'] = user[0]
            flash('Login Successful!')
            return redirect(url_for('index'))
        else:
            return render_template('login.html', error='Invalid
username or password!')
```

Preventing CSRF Attacks

CSRF attacks are possible on this website as the edit route in the server.py file allows actions without verifying if the request comes from a trusted source. An attacker can create a malicious webpage or send an email to a logged-in user, tricking them into editing someone else's comment without their knowledge through a maliciously crafted URL.

To fix this possibility of CSRF attacks there are two options: we can either use CSRF tokens or we can restrict dangerous actions to POST requests only and check user authentication.

Using CSRF tokens in server.py:

```
#Flask doesn't protect against CSRF by default so use Flask-WTF
from flask_wtf.csrf import CSRFProtect
```

```
csrf = CSRFProtect(app) # Enable CSRF protection
```

After making this change to the server.py file, we have to modify the edit.html file to include the CSRF tokens at point of modifications or deletions to comments:

```
<body>
  {% for entry in entries %}
    <form action="/login/ok/{{ entry.id }}" method="POST">
      <input type="hidden" name="csrf_token" value="{{
csrf_token() }}">
      <dl>
        <dt>Provide your Feedback:</dt>
        <dd><textarea name="text" rows="5" cols="40">{{
entry.feedback }}</textarea></dd>
        <dd><input type="submit" value="Share"></dd>
      </dl>
    </form>

    <form action="/login/delete/{{ entry.id }}" method="POST">
      <input type="hidden" name="csrf_token" value="{{
csrf_token() }}">
      <input type="submit" name="delete" value="Delete">
    </form>
  {% endfor %}
</body>
```

After this, only forms with a valid CSRF token can execute requests. This ensures only legitimate requests from the user's browser are accepted and attackers can't forge requests without the token.

We can also change the edit route to only accept POST and check user authentication:

```

@app.route('/login/edit/<id>', methods=['POST'])
def edit_entry(id):
    if 'user' not in session:
        abort(403) # Forbidden

    error = None
    cur = g.db.execute("SELECT id, feedback, author FROM feedbacks
WHERE id = ?", (id,))
    a = cur.fetchall()

    entries = [dict(id=row[0], feedback=row[1], author=row[2]) for
row in a]

    if request.method == 'POST':
        new_feedback = request.form['feedback']
        g.db.execute("UPDATE feedbacks SET feedback = ? WHERE id =
?", (new_feedback, id))
        g.db.commit()
        return redirect(url_for('index'))

    return render_template('edit.html', Name=name, entries=entries,
user_session=session['user'][0])

```

We have now prevented CSRF and unauthorised access.

Preventing Command Injection Attacks

The website is also susceptible to command injection attacks, as the /dns route directly passes user input into a system command:

```

cmd = 'nslookup ' + name
output = subprocess.check_output(cmd, shell=True, text=True)

```

This means an attacker can execute arbitrary commands through payloads like “www.google.com; rm -rf /” which would delete all files on the server. To fix this, we can use shlex.quote() to Sanitize Input, instead of processing raw user input:

```

import shlex

@app.route("/dns", methods=['GET'])
def page():
    name = request.args.get('hostname')
    output = None

    if name:
        safe_name = shlex.quote(name) # Sanitizes input
        cmd = ['nslookup', safe_name]
        try:
            output = subprocess.check_output(cmd, text=True)

```



```
except subprocess.CalledProcessError:
    output = "Invalid hostname or error occurred"

return render_template('dns.html', output=output)
```

This prevents arbitrary command execution. To make this feature of the website even safer, we can also ensure that the server runs as a restricted user with no root access, this will limit damage if any attack is successful.